

Shopify Integration

See also: [TESTING_GUIDE.md](#)

1. Overview

1.1 What this integration does

Shopify Integration connects the **Shopify Admin GraphQL API** to the **Newo Platform** and is currently focused on one main customer journey:

- periodically load the full product catalog into a customer attribute
- check product availability
- create a draft order
- give the customer an invoice link
- update or cancel the draft order if needed
- after payment, move the booking state from `draft` to `paid`
- check order/payment status when the customer asks

The integration stores the customer-facing order state in the `bookings` persona attribute.

1.2 Current pipeline

The current logic is built around draft order payment:

1. **CollectProductsFlow** On `session_started` or a manual trigger, the integration refreshes `shopify_products_catalog` and stores the normalized full product catalog there.
2. **ProductAvailabilityFlow** The agent searches for a product and checks inventory.
3. **CreateDraftOrderFlow** The agent creates a draft order in Shopify, stores a record in `bookings`, and marks it as:
 - `status = "draft"`
 - `is_draft = "true"`
 - `payment_status = "pending"`
4. After the draft order is created successfully, the agent:
 - receives `invoice_url`
 - receives prompt context through `set_prompt_custom_section`
 - must tell the customer that the order is **not paid yet**
5. **UpdateDraftOrderFlow** While the order is unpaid, the agent can:
 - change line items
 - change the note
 - change the shipping address
 - apply a discount
6. **CancelDraftOrderFlow** While the order is unpaid, the draft order can be cancelled.
7. **CheckOrderFlow** If the customer asks for status or says they have already paid the invoice, the flow shows the order/payment/fulfillment state and syncs `bookings` to `paid` if Shopify already sees the order as paid.

1.3 Supported capabilities

Capability	Supported	Notes
OAuth setup	✓	OAuth only, no custom app path
Admin GraphQL requests	✓	Business logic only through the Admin GraphQL API
Full catalog export into an attribute	✓	Through CollectProductsFlow into shopify_products_catalog
Order lookup	✓	Order status, payment status, fulfillment, tracking
Product availability lookup	✓	Product search + inventory levels
Draft order creation	✓	Stored in bookings
Draft order update	✓	Including discount logic
Draft order cancellation	✓	Through draftOrderDelete
Draft -> paid sync through manual order lookup	✓	Through CheckOrderFlow
Customer sync flow	✗	Removed as unnecessary
Store locations flow	✗	Removed as unnecessary
Storefront API	✗	Fully removed

2. Requirements

2.1 Main project attributes

- shopify_oauth_code
- shopify_newo_webhook_api_key (read-only)
- shopify_feature_order_status_enabled
- shopify_feature_product_availability_enabled
- shopify_feature_create_draft_order_enabled
- shopify_feature_update_draft_order_enabled
- shopify_feature_cancel_draft_order_enabled
- shopify_feature_customer_profile_enabled
- shopify_feature_return_exchange_enabled
- shopify_feature_product_catalog_refresh_enabled
- shopify_shop_id
- shopify_client_id
- shopify_client_secret
- shopify_setup_scenarios
- shopify_access_token (hidden)
- shopify_refresh_token (hidden)
- shopify_base_url (hidden)
- shopify_api_version (hidden)

- `shopify_products_catalog` (hidden)
- `shopify_products_catalog_buffer` (hidden)
- `shopify_products_catalog_updated_at` (hidden)
- `shopify_products_catalog_update_interval` (hidden)

The OAuth Redirect URL is fixed in code as `https://static.newo.ai/oauth/index.html` and is not configured through a project attribute.

2.2 Recommended Shopify scopes

- `read_customers`
- `read_orders`
- `read_all_orders`
- `read_products`
- `read_inventory`
- `read_draft_orders`
- `write_draft_orders`
- `read_discounts`

3. Architecture

3.1 Active flows

Flow	Trigger(s)	Purpose
SetupFlow	<code>project_publish_finish</code> , <code>result_success</code> on <code>shopify_connector</code>	Set up attributes, connectors, OAuth token exchange, and tool registration
CanvasSetupFlow	<code>gm_workflow_builder_canvas_built</code>	Adds Canvas intents/scenarios for the current Shopify tools
UtilsFlow	<code>shopify_execute_request</code> , <code>HTTP result_success/result_error</code>	Shared HTTP/GraphQL transport and token handling
CollectProductsFlow	<code>session_started</code> , <code>shopify_collect_products</code> , <code>shopify_collect_products_webhook</code>	Loads the full product catalog into a customer attribute
ProductAvailabilityFlow	<code>shopify_product_availability_event</code>	Searches products and inventory
CreateDraftOrderFlow	<code>shopify_draft_order_event</code>	Creates a draft order
UpdateDraftOrderFlow	<code>shopify_update_draft_order_event</code>	Updates a draft order, including discount updates
CancelDraftOrderFlow	<code>shopify_cancel_draft_order_event</code>	Cancels a draft order
CheckOrderFlow	<code>shopify_order_status_event</code>	Checks order/payment/fulfillment status

3.2 Transport pattern

Almost all business flows work the same way:

1. The entry skill parses `payload.shopify`
2. A GraphQL query/mutation is built
3. `shopify_execute_request` is sent
4. `UtilsFlow` calls the Shopify Admin GraphQL endpoint
5. The response returns as `shopify_result_success` or `shopify_result_error`
6. Flow-specific success/error skills update state and send `urgent_message`

3.3 Booking state storage

The order state is stored in the `bookings` persona attribute.

When a draft order is created or updated, the integration stores a record with `meta` containing:

- `source`
- `booking_id`
- `booking_reference`
- `draft_order_id`
- `draft_order_name`
- `invoice_url`
- `status`
- `is_draft`
- `payment_status`
- `booking_parameters`

After payment, the same record is extended or updated with:

- `order_id`
- `order_name`
- `financial_status`
- `order`

3.4 Draft-to-paid order linking

To reliably update the correct `bookings` record later, the integration writes service markers into the draft order note:

- `TWNM_BOOKING_REFERENCE`
- `TWNM_PERSONA_USER_ID`

`CheckOrderFlow` then reads those markers from matching orders.

The booking is moved from `draft` to `paid` based on those markers.

4. Flow Overview

4.1 SetupFlow

What it does

- creates `shopify_connector`
- creates `shopify_token_connector`
- prepares project attributes
- exchanges `shopify_oauth_code` for tokens when present

- registers Shopify tools

Main skills

- SetupSkill
- _getTokensSkill
- _getTokensSuccessSkill
- _setupToolBooking

Result

- ready attributes
- ready connectors
- registered custom tools

4.2 UtilsFlow

What it does

- sends all GraphQL requests to Shopify
- sets X-Shopify-Access-Token
- routes success/error back to the Shopify flow
- handles token response and refresh path

Main skills

- ExecuteRequestSkill
- ResultSuccessSkill
- ResultErrorSkill
- TokenResultSuccessSkill
- _executeRequestSkill
- _refreshTokenSkill

4.3 CollectProductsFlow

What it does

- loads the full product catalog through Shopify Admin GraphQL products
- paginates through the full catalog
- normalizes products and variants
- stores the result in shopify_products_catalog
- updates shopify_products_catalog_updated_at

Trigger

- session_started
- shopify_collect_products
- shopify_collect_products_webhook

Key attributes

- shopify_products_catalog
- shopify_products_catalog_updated_at
- shopify_products_catalog_update_interval

4.4 ProductAvailabilityFlow

What it does

- accepts `product_search_query`
- searches products
- fetches inventory levels
- returns inventory by location

Trigger

- `shopify_product_availability_event`

Test webhook

- `shopify_product_availability_test_webhook`

4.5 CreateDraftOrderFlow

What it does

- accepts customer/product input
- finds the customer
- finds the product / variant
- creates a draft order
- stores it in `bookings`
- sets the `draft/pending` state
- puts invoice context into the prompt
- asks the agent to tell the customer the invoice link and that payment is still pending

Trigger

- `shopify_draft_order_event`

Test webhook

- `shopify_create_draft_order_test_webhook`

4.6 UpdateDraftOrderFlow

What it does

- finds the existing draft order
- fetches current line items / note / shipping
- updates the draft order
- applies a discount within the same flow when needed
- updates the `bookings` record
- preserves the invoice link and pending status

Trigger

- `shopify_update_draft_order_event`

Test webhook

- `shopify_update_draft_order_test_webhook`

4.7 CancelDraftOrderFlow

What it does

- finds the relevant draft order
- deletes it through Shopify

- removes the record from `bookings`
- updates prompt context so the invoice link is no longer used

Trigger

- `shopify_cancel_draft_order_event`

Test webhook

- `shopify_cancel_draft_order_test_webhook`

4.8 CheckOrderFlow

What it does

- searches Shopify customers by email/phone
- fetches recent orders
- returns:
 - financial status
 - fulfillment status
 - total
 - tracking
- updates `bookings` to `paid` if it finds a paid order with our `booking_reference`

Trigger

- `shopify_order_status_event`

Test webhook

- `shopify_check_order_test_webhook`

5. Custom Tools and Canvas

The agent currently registers only the user-facing tools required by the current pipeline:

- `shopify_product_availability_event`
- `shopify_draft_order_event`
- `shopify_update_draft_order_event`
- `shopify_cancel_draft_order_event`
- `shopify_order_status_event`

Canvas scenarios are also reduced to the same model:

- product availability
- draft order operations
- order status

6. What to Test

Minimum smoke path:

1. `shopify_product_availability_test_webhook`
2. `shopify_create_draft_order_test_webhook`
3. Verify that `bookings` contains a record with `draft` status
4. `shopify_update_draft_order_test_webhook`
5. Pay the invoice or use an already-paid order for the same customer

6. `shopify_check_order_test_webhook`
7. Verify that the `bookings` record changed to `paid` if Shopify already sees the order as paid
8. `shopify_cancel_draft_order_test_webhook` for a separate draft

7. Current Limitations

- Shopify webhook HMAC validation was not added in this iteration
- Automatic invoice sending through a separate Shopify mutation is not implemented
- Customer communication after draft order creation is built through `urgent_message` + `set_prompt_custom_section`
- Automatic webhook sync from paid order no longer exists; the `draft -> paid` transition currently happens through `CheckOrderFlow`
- `CheckOrderFlow` syncs to paid only for orders whose note contains our service markers

8. Summary

The integration is no longer a set of scattered Shopify CRUD flows. It is now a narrow, clear pipeline:

- find a product
- create a draft order
- guide the customer to payment
- update or cancel the draft if needed
- after payment, move booking state to `paid` through `CheckOrderFlow`
- show the current order status when the customer asks